

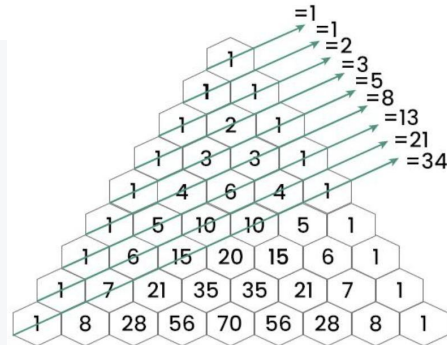
递归和动态规划的分析积累

爬楼梯问题：假设你正在爬一个楼梯。它总共有 n 阶台阶。每次你只能爬 1 阶或者爬 2 阶。问:你有多少种不同的方法可以爬到楼顶?

答: 这是一个经典的**动态规划**问题(动态规划是解决优化问题的强大工具, 动态规划 = 分解问题 + 存储结果 + 避免重复计算)! 也是一种**斐波那契数列**问题:

动态规划解法：(第 40 页是递归解法)

```
1 #include <stdio.h>
2
3 int climbStairs(int n) {
4     if (n <= 2) return n;
5
6     int dp[n+1];
7     dp[1] = 1;
8     dp[2] = 2;
9
10    for (int i = 3; i <= n; i++) {
11        dp[i] = dp[i-1] + dp[i-2];
12    }
13
14    return dp[n];
15 }
16
17 int main() {
18     int n = 5;
19     printf("爬%d阶楼梯有%d种方法\n", n, climbStairs(n));
20     return 0;
21 }
```



斐波那契数列

现在请你帮我分析动态规划问题和递归问题的区别和共同点，我要做个笔记

动态规划 vs 递归：区别与共同点

核心区别

特性	递归	动态规划
重复计算	重复计算相同子问题	存储子问题结果，避免重复计算
时间复杂度	指数级 $O(2^n)$	多项式级 $O(n^2)$ 或 $O(n)$
空间复杂度	调用栈深度 $O(n)$	额外存储空间 $O(n)$
实现方式	自顶向下	自底向上或记忆化
适用问题	简单、无重叠子问题	有重叠子问题的优化问题

详细对比

1. 递归问题

```
1 // 递归实现 (重复计算)
2 int fib_recursive(int n) {
3     if (n <= 1) return n;
4     return fib_recursive(n-1) + fib_recursive(n-2); // 重复计算
5 }
```

例子 1

```
1 int fib(int n) {
2     if (n <= 1) return n;
3     return fib(n-1) + fib(n-2); // 重复计算很多次!
4 }
```

例子 2

特点：直接、直观

问题：大量重复计算

时间复杂度： $O(2^n)$

2. 动态规划

```
1 // 动态规划实现 (避免重复计算)
2 int fib_dp(int n) {
3     int dp[n+1];
4     dp[0] = 0; dp[1] = 1;
5     for (int i = 2; i <= n; i++) {
6         dp[i] = dp[i-1] + dp[i-2]; // 只计算一次
7     }
8     return dp[n];
9 }
```

特点：存储中间结果

优势：时间复杂度 $O(n)$

空间复杂度： $O(n)$

共同点：①**问题分解：**都将大问题分解为子问题②**最优子结构：**大问题的解依赖子问题的解

③**递推关系：**都有明确的状态转移方程④**边界条件：**都需要定义基础情况

何时使用哪种方法？

选择递归的情况：问题规模小 无重叠子问题 代码简洁性更重要 深度优先搜索等场景

选择动态规划的情况：问题规模大 有重叠子问题 需要优化性能 求解最优化问题

实际应用对比

问题	递归解法	动态规划解法
爬楼梯	$O(2^n)$	$O(n)$
硬币找零	$O(2^n)$	$O(n \times m)$
最长递增子序列	$O(2^n)$	$O(n^2)$

学习建议：①**先理解递归：**掌握问题分解和边界条件②**识别重叠子问题：**判断是否需要动态规划③**掌握状态定义：**明确 dp 数组含义④**建立状态转移方程：**找到递推关系⑤**优化空间：**考虑滚动数组等技巧

⑥**避免重复计算：**使用记忆化或存储中间结果

⑦**边界条件：**明确基础情况

总结：动态规划是递归的优化版本，理解两者的关系有助于掌握算法设计的精髓！